

Video Summary

This video explains how Python variables work with memory. Variables are references to objects, `id()` shows object identity, Python may reuse common objects for efficiency, and unreferenced objects are cleaned by the Garbage Collector.

Source video: <https://www.youtube.com/watch?v=nEfoSZKp3S8&list=PLsyebzWxl7omDoEYrrf3oXvXxa6MPgek&index=12>

1 Variables Are References

A variable is not a box that directly contains a value. It is a name that points to an object stored somewhere in memory.

name**Variable name**

A label used by your code to reach an object.

obj**Object**

The actual value stored in memory, such as 10 or Python.

id**Identity**

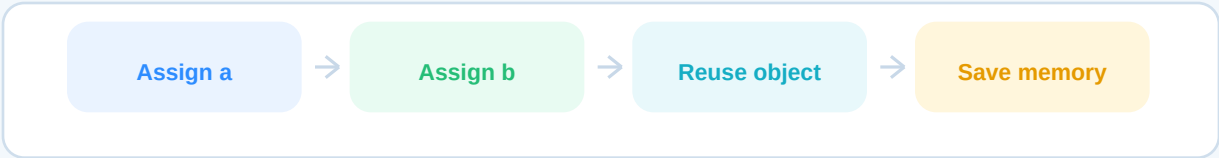
`id()` shows the unique identity of an object during a run.

```
>>> x = 10
>>> id(x)
140732882490440 # example address-like identity
>>> y = x
>>> id(y) == id(x)
True
# x and y reference the same object
```

references and id()

2 Memory Efficiency

Python can reuse existing objects instead of creating a duplicate every time. This is common for small integers and short strings.



same object example

```
>>> a = 10
>>> b = 10
>>> id(a) == id(b)
True
>>> name1 = "python"
>>> name2 = "python"
>>> id(name1) == id(name2)
True
```

Case	Why Python may reuse it	Beginner meaning
small integer	Frequently used values are cached.	Same value may share one object.
short string	Common identifiers are often interned.	Same text may share one object.
large object	Reuse is selective, not universal.	Do not depend on same identity.

3

Rebinding Variables

Assigning a new value does not edit the old object. The variable name simply starts pointing to a different object.

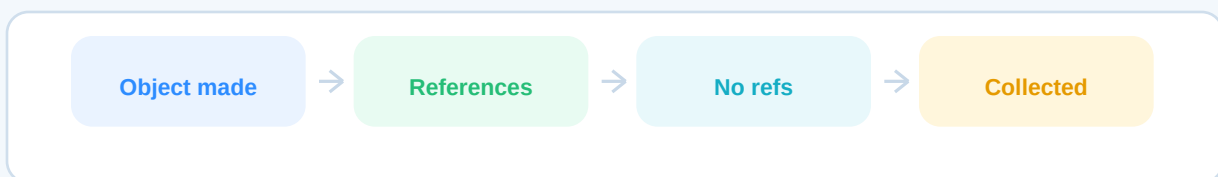
```
rebinding example

>>> x = 10
>>> old_id = id(x)
>>> x = 20
>>> id(x) == old_id
False
# x now references a different integer object
```

4

Garbage Collection

When no variable or container refers to an object, that object is no longer useful to the program. Python can reclaim that memory automatically through its Garbage Collector.



Memory cleanup rule

You usually do not manually free normal Python objects. Keep references only as long as you need them; Python handles unreachable objects in the background.

5

String Interning

String interning is an optimization where Python stores selected strings once and reuses that same object when possible. It is especially common for short strings that look like identifiers.

str**Short strings**

Common short strings are good candidates for reuse.

pool**Intern pool**

Python can keep one copy and point names to it.

opt**Optimization**

Interning is for efficiency, not program logic.



interning example

```
>>> s1 = "navin"
>>> s2 = "navin"
>>> id(s1) == id(s2)
True
>>> s1 is s2
True
# short repeated strings may share identity
```

Important distinction

Use `==` to compare values. Use `is` only when you specifically need identity comparison, such as checking whether two names point to the same object.

6 Limitations of Caching

Python memory reuse is smart but selective. It is generally reserved for small integers, common short strings, and values where reuse helps overall efficiency.

Value type	Likely behavior	Rule for beginners
small integers	Often cached and reused.	Same value may have same id.
short strings	May be interned and reused.	Same text may have same id.
large values	Reuse is not guaranteed.	Do not rely on identity.
complex objects	Usually separate objects when created separately.	Compare contents with ==.

```
value vs identity

>>> a = 10; b = 10
>>> a == b, a is b
(True, True)
# cached values may share identity
>>> [1, 2] == [1, 2]
True
>>> [1, 2] is [1, 2]
False
```

Beginner rule

For normal data comparison, prefer ==. Treat id() and is as tools for understanding references and object identity.

7

Practice Code

Run this small script to observe identity, shared references, rebinding, and garbage collection support.

```
import gc

a = 10
b = 10
print(id(a) == id(b))

name1 = "python"
name2 = "python"
print(name1 is name2)

values = [1, 2, 3]
alias = values
print(id(values) == id(alias))

values = None
alias = None
print(gc.collect())
```

Idea	What to watch	Meaning
<code>id(a) == id(b)</code>	Small integer object reuse.	May print True.
<code>name1 is name2</code>	Short string interning.	May print True.
<code>gc.collect()</code>	Manual collection request.	Returns objects collected.

Day 13/55 takeaway

Python variables are references to objects. `id()` shows identity, Python reuses selected common objects for memory efficiency, and the Garbage Collector cleans unreachable objects automatically.